UT3
Manejo de conectores JDBC

ACCESO A DATOS

2º DAM

Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial 4.0 Internacional.

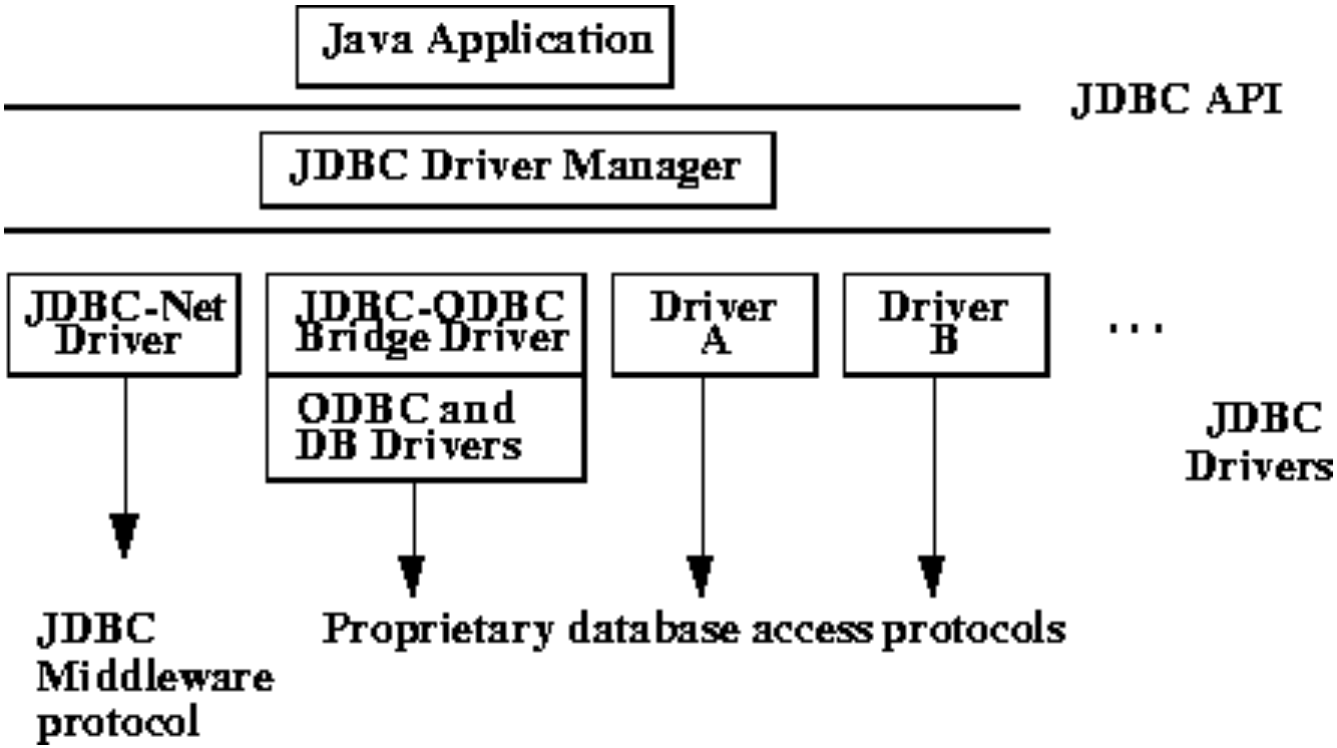
Contenidos

- Productos JDBC. Tipos de Drivers
- JDBC – Establecer Conexiones
- JDBC – Realizar consultas SQL
- JDBC – PreparedStatements
- JDBC – Manejar Excepcions SQL
- JDBC – Recuperar y Modificar Valores de un ResultSet
- JDBC – Transacciones
- Arquitectura en capas
  - Patrón DAO

Productos JDBC. Tipos de Drivers

- JDBC es el API de Java que nos permite acceder a datos en formato de tabla, en especial a datos almacenados en BD relacionales.
- JDBC incluye 4 componentes:
  - JDBC-API: permite a las aplicaciones a través del lenguaje Java ejecutar sentencias SQL, recuperar datos y propagar cambios en un origen de datos (BD, FS)
    - Paquetes java.sql y javax.sql incluidos en Java SE y Java EE
  - JDBC Driver Manager: es el backbone de JDBC. Es la clase que nos pemite conectar una apliación Java a un driver JDBC
  - JDBC Test Suite: permite determinar que drivers puede ejecutar la aplicación
  - JDBC-ODBC Bridge: Puente que nos permite acceder a driver ODBC via JDBC
    - El driver ODBC debe estar en cada cliente que se vaya a usar, no lo proporciona Java

Productos JDBC. Tipos de Drivers

- 

Productos JDBC. Tipos de Drivers

- **Necesitamos instalar un driver JDBC para el sistema gestor de BD que vayamos a utilizar. Este driver nos lo proporciona el vendedor.**
- Existen 4 tipos de implementaciones de drivers JDBC:

Tipo 1: Drivers tipo JDBC-ODBC. Implementan un “mapeo” entre JDBC y otro API, por ejemplo ODBC. Son dependientes de una librería nativa por lo que tienen portabilidad limitada.

Sólo usarlos si el vendedor no proporciona un driver JDBC. Oracle por ejemplo no soporta JDBC-ODBC.

Tipo 2: Drivers escritos parte en Java y parte en código nativo. Son dependientes de una librería nativa por lo que tienen portabilidad limitada. Oracle OCI (Oracle Call Interface) driver es de este tipo

Productos JDBC. Tipos de Drivers

Tipo 3: Drivers que usan un cliente Java puro y se comunican con un middleware utilizando un protocol independiente de la BD. El middleware se comunica con el origen de datos

Tipo 4: Drivers que usan un cliente Java puro y que implementan un protocolo específico para un origen de datos. El cliente se comunica directamente con el origen de datos.

- Java DB incorpora 2 drivers de tipo 4.
- MySQL Connector/J is un driver de tipo 4.

Instalar un driver JDBC consiste en copiar el driver al equipo y añadir su ubicación al class path. La mayoría de los drivers que no son de tipo 4 también requieren instalar un API del lado del cliente.

Utilidades.java
PruebaUtilidades.java
mysql-properties.xml

JDBC - Establecer conexiones

Para crear una conexión necesitamos usar una de las 2 clases siguientes:

- DriverManager: Clase implementada por completo que conecta una aplicación con un origen de datos especificado por un URL. Carga automáticamente los drivers JDBC 4.0 que encuentre en class path.

- DataSource: Interfaz que oculta los detalles de bajo nivel del origen de datos a la aplicación. Proporciona características avanzadas como “pool de conexiones” y transacciones distribuidas.
  - Se utiliza en aplicaciones Java EE.
  - Más complejo de utilizar

Utilidades.java
PruebaUtilidades.java
mysql-properties.xml

Actividad

- Lee el código de las 3 clases anteriores. Asegúrate de entenderlo
- Cambia los valores de mysql-properties.xml a valores adecuados para tu conexión
- Ejecuta PruebaUtilidades.java y verifica que se realizar la conexión a BD correctamente
- Modifica Utilidades.java para que el archivo de propiedades sea una constante de la clase.

Cafes.java (método verTabla)
PruebaCafes.java

JDBC - Realizar consultas SQL

Para procesar cualquier sentencia SQL con JDBC hay que seguir los siguientes pasos:

- Establecer una conexión (Connection)
- Crear una sentencia (Statement)
- Ejecutar la consulta
  - **execute**: Devuelve true si el primer objeto que devuelve la consulta es un ResultSet. Utilizarlo cuando la consulta puede devolver uno o más ResultSets. Llamando repetidamente a Statement.getResultSet podemos ir recuperando los objetos ResultSet
  - **executeQuery**: Devuelve un Objeto ResulSet.
  - **executeUpdate**: Devuelve un entero que respresenta el número de filas afectadas por la sentencia. Utilizar con sentencias SQL INSERT, DELETE, UPDATE.
- Procesar el resultado (ResultSet)
- Cerrar la conexión (Connection)

Actividad

- Lee el código de las 2 clases anteriores. Asegúrate de entenderlo
- Ejecuta PruebaCafes.java y verifica que funciona correctamente
- Añade los siguientes métodos en la clase Cafes.java:
  - Un método que busque un café por nombre y muestre los datos de ese café
  - Un método que dado un nombre de café borre los datos de ese café
  - Un método que dados los datos de un café inserte una fila
- Después añade llamadas a estos métodos en PruebaCafes.java para comprobar que funcionan correctamente

Cafes.java (método actualizarVentasCafe
PruebaCafes.java

JDBC - Prepared Statements

Una sentencia PreparedStatement es una especialización de Statement con 2 ventajas fundamentales:

- La consulta se envía precompilada al Gestor de BD que sólo tendrá que ejecutarla y por tanto es más eficiente
- Permite realizar consultar parametrizadas:
  - La construcción de consultas con parámetros se simplifica mucho
  - Podemos utilizar la misma consulta con distintos valores

Actividad

- Reescribe los 3 métodos de la actividad anterior para utilizar preparedStatements en lugar de Staments y comprobar lo eficiente que es para realizar consultas parametrizadas

método printSQLException

JDBC - Manejo de Excepciones SQL

- JDBC lanza una SQLException cuando ocurre un error al interaccionar con un origen de datos
- Una SQLEXception contiene la siguiente información que nos puede ayudar a determinar la causa del error:
  - Una descripción textual
  - Un código de estado SQL según es el estándar ISO/ANSI
  - Un código de error numérico dependiente del origen de datos subyacente
  - Una causa. Es posible que una SQLException sea causada por otros objetos.
  - Una referencia a cualquier otra excepción que se produzca en cadena
- Es conveniente obtener esta información de SQLException para depurar errores de fácilmente

Actividad

- Modifica la clase Cafes.java para que la consulta de verTabla se realice sobre una tabla que no exista. Vamos a forzar una excepción
- Ejecuta la clase PruebaCafes.java.
- Comprueba la salida por pantalla. Ahora modifica este método para que cuando hagas un catch para SQLException llame al método printSQLException de Utilidades.java
- Comprueba la salida por pantalla. Observa como ahora la información es mucho más completa.
- Modifica en la clase Cafes.java todos los catch para SQLException para que utilicen el nuevo método printSQLException de Utilidades.java
- Piensa en un aplicación real con interfaz gráfica (para nosotros sería PruebaCafes.java). En nuestro ejemplo estamos mostrando los errores por pantalla pero en una aplicación real se escribirían en un log por lo que el error no llega a la GUI y no podemos informar al usuario ¿Cómo lo solucionamos? ¿lanzamos SQLEXception en Cafes.java en vez de usar catch?¿Crees que sería adecuado que la GUI manejase SQLException? ¿Y si queremos cambiar el origen de datos? Piensa como resolverlo.

Actividad

- Para practicar
  - Ejercicios1.pdf

JDBC - Recuperar y Modificar Valores de un ResultSet

- Un ResultSet es una tabla que almacena los dados generados al ejecutar una consulta
- Para acceder a los datos se utiliza un cursor (puntero a una fila del ResulSet, su posición inicial es antes de la primera fila)
- Con el método next movemos el cursor a la siguiente fila
- La interfaz ResultSet declara métodos getter para recuperar valores de columnas de la fila actual según el tipo de datos (getLong, getBoolean)
- Se pueden recuperar los valores utilizando un índice númerico o el alias (nombre). Se aconseja:
  - Utilizar índice numérico
  - Leer las columnas de izquierda a derecha
  - Leer cada columna una única vez

Actividad

- Consulta en al API de Java los métodos getter de la interfaz ResultSet http://docs.oracle.com/javase/8/docs/api/
- Consulta la clase Cafes.java, ¿Cómo se recuperan los valores? ¿Se siguen los consejos de la transparencia anterior? Realiza los cambios necesarios para que se cumplan todas las recomendaciones

JDBC - Recuperar y Modificar Valores de un ResultSet

Concurrencia de ResulSet

Determina si el ResultSet es actulizable o no

- **CONCUR_READ_ONLY:** No se puede actualizar**.**
- **CONCUR_UPDATABLE:** Se puede actualizar**.**
- Por defecto ResulSet es de **CONCUR_READ_ONLY**

método modificarPrecios de Cafes.java

JDBC - Recuperar y Modificar Valores de un ResultSet

Tipos de ResulSet

- **TYPE_FORWARD_ONLY**: El cursor solo puede avanzar hacia delante. Desde antes de la primera fila hasta después de la última.
- **TYPE_SCROLL_INSENSITIVE**: El cursor se puede mover hacia delante y hacia atrás de forma relativa a la posición actual. También se puede mover a una posición absoluta. El ResulSet es insensible a cambios que se hagan en el origen de datos mientras permanence abierto.
- **TYPE_SCROLL_SENSITIVE**: El cursor se puede mover hacia delante y hacia atrás de forma relativa a la posición actual. También se puede mover a una posición absoluta. El ResulSet refleja los cambios que se hagan en el origen de datos mientras permanence abierto.
- Por defecto ResulSet es de tipo **TYPE_FORWARD_ONLY**.

JDBC - Recuperar y Modificar Valores de un ResultSet

métodos modificarPrecios e insertarFila e
Cafes.java

Métodos para mover el cursor

- **next**: Mueve el cursor una fila hacia delante. Devuelve true si se posciona en una fila y false si después de la última fila. (El único que se puede utilizer con)
- **previous**: Mueve el cursor una fila hacia atrás. Devuelve true si se posciona en una fila y false si antes de la última fila.
- **first**: Mueve el cursor a la primera fila. Devuelve true si se posciona en la primera fila y false si no hay filas en el ResulSet .
- **last**: Mueve el cursor a la última fila. Devuelve true si se posciona en la última fila y false si no hay filas en el ResulSet .
- **beforeFirst**: Mueve el cursor antes de la primera fila. Sin efecto si el ResulSet no tiene filas
- **afterLast**: Mueve el cursor después de la última fila. Sin efecto si el ResulSet no tiene filas
- **relative(int rows):** Mueve el cursor relativa a su posición actual.
- **absolute(int row):** Posiciona el cursor en la fila especificada de forma absoluta.

métodos actualizarVentas en Cafes.java

JDBC - Transacciones

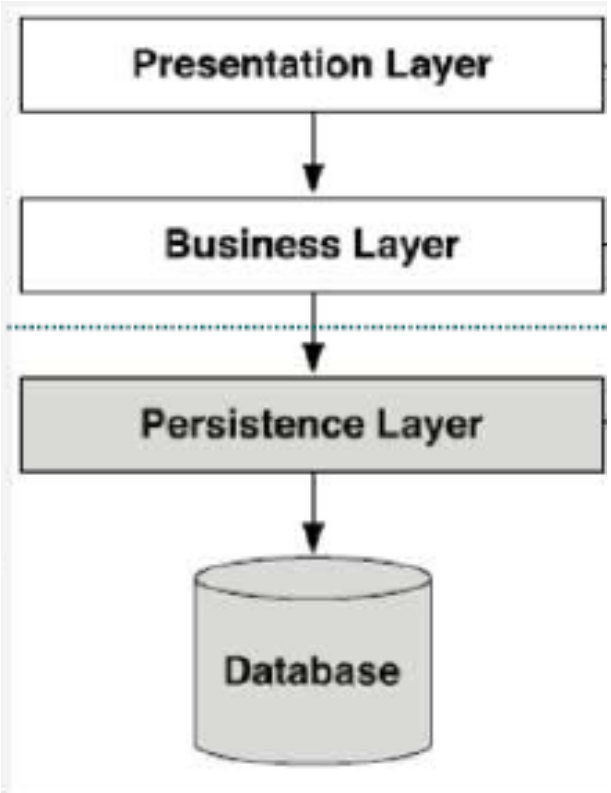
- Una transacción es un conjunto de operaciones que queremos que se realicen de forma atómica, es decir o se realizan todas con éxito ó ninguna. Esto implica que si alguna de las operaciones se realiza y se produce un error habrá que deshacer los cambios
- En nuestro ejemplo de Cafes nos interesa que cuando actualicemos las ventas se actualice el total también
- No nos interesa que se actualice una venta sin actualizar el total, esto es una transacción.
- Para realizar una transacción en java:
  - Deshabilitar el autocommit (Por defecto cualquier sentencia SQL que ejecutemos tiene efecto automático en la BD)
  - Realizar un commit explícitamente cuando realicemos todas las operaciones de nuestra transacción
  - Realizar un rollback (deshacer operaciones parciales) si se produce un error en BD

Actividad

- Para practicar
  - Ejercicios2.pdf

Arquitectura en Capas

- La arquitectura define los componentes hardware y software de una aplicación y cómo se relacionan entre ellos.
- Las aplicaciones web tienen una arquitectura de capas.
- Las capas se comunican a través de interfaces con la inferior
- Así conseguimos que sean independientes y reemplazables



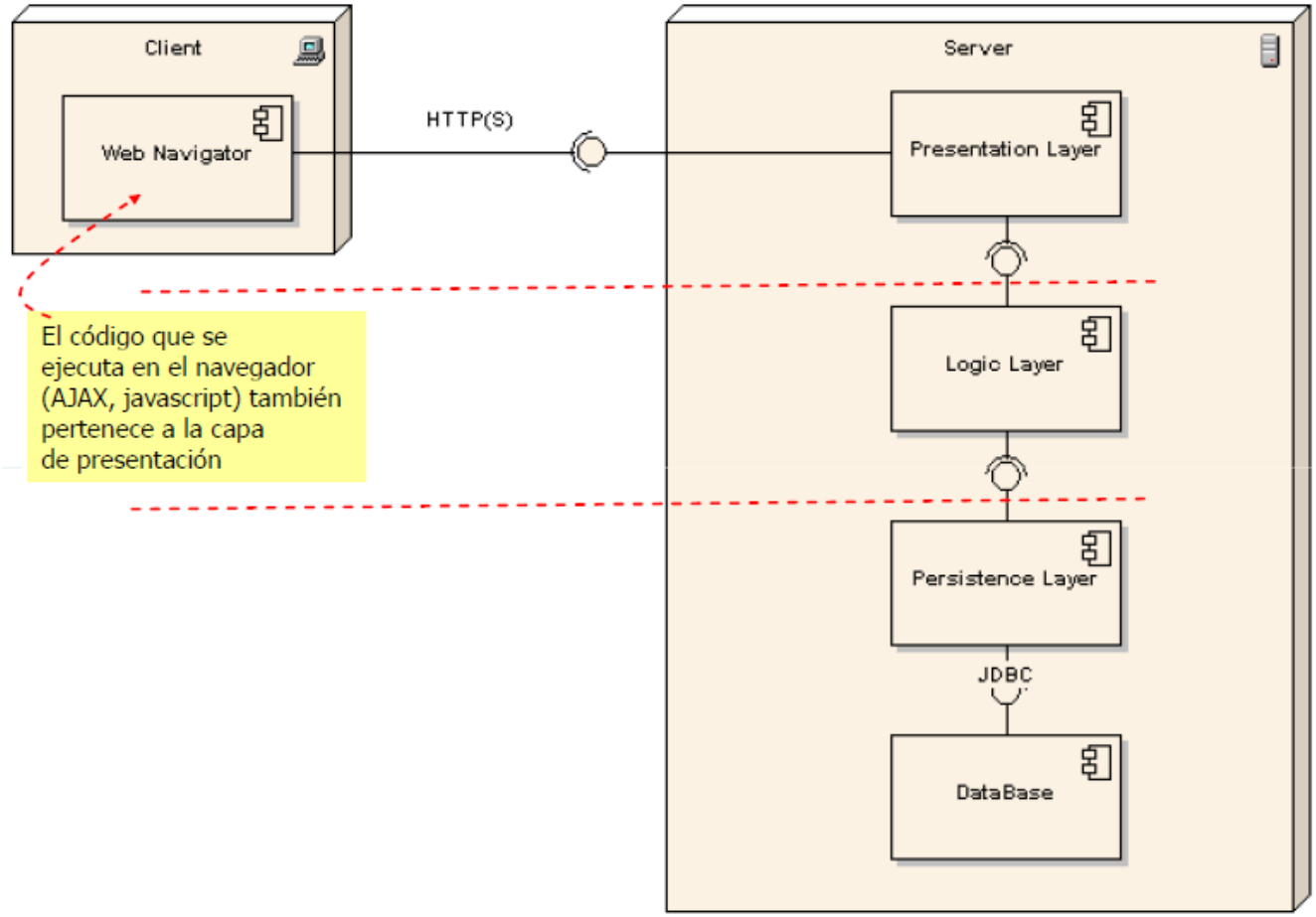
Arquitectura en Capas

- **Capa de presentación (GUI):**
  - Se encarga de la interacción con el usuario: solicitar, mostrar, formatear, internacionalización de datos
  - Informar adecuadamente de errores de la aplicación
  - En una aplicación web parte esté en el servidor y parte es resuelta por el navegador (AJAX)
  - Es habitual utilizar un patrón **MVC (Model View Controller, ej: framework struts)**
- **Capa de negocio**
  - Resuelve las funcionalidades para las que se ha diseñado la aplicación ofreciéndolas como servicios: Clases con métodos que implementan estas funciones (ej: un servicio web)
  - Para ello necesitamos objetos que representen las entidades de nuestra aplicación
  - Para ello accede a la capa de persistencia y envía/recibe datos de la capa de presentación siendo independiente de ambas

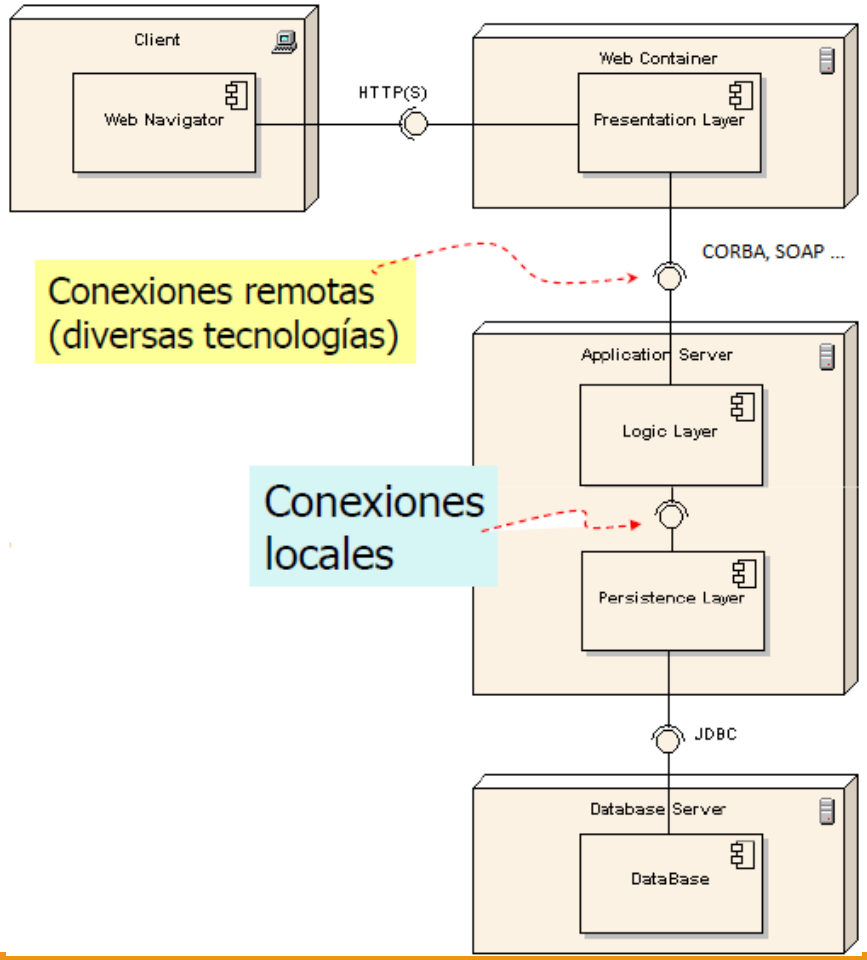
Arquitectura en Capas

- **Capa de persistencia**
  - La persistencia de datos puede implementarse con múltiples y diferentes tecnologías: ficheros, gestores de BD relacionales diferentes: Oracle, mysql… , BD no relacionales
  - La implementación de la persistencia tiene que ser “transparente” para las capas de negocio y presentación
  - Esta independencia de la implementación concreta se consigue mediante **un patrón DAO**

Arquitectura en Capas



Arquitectura en Capas

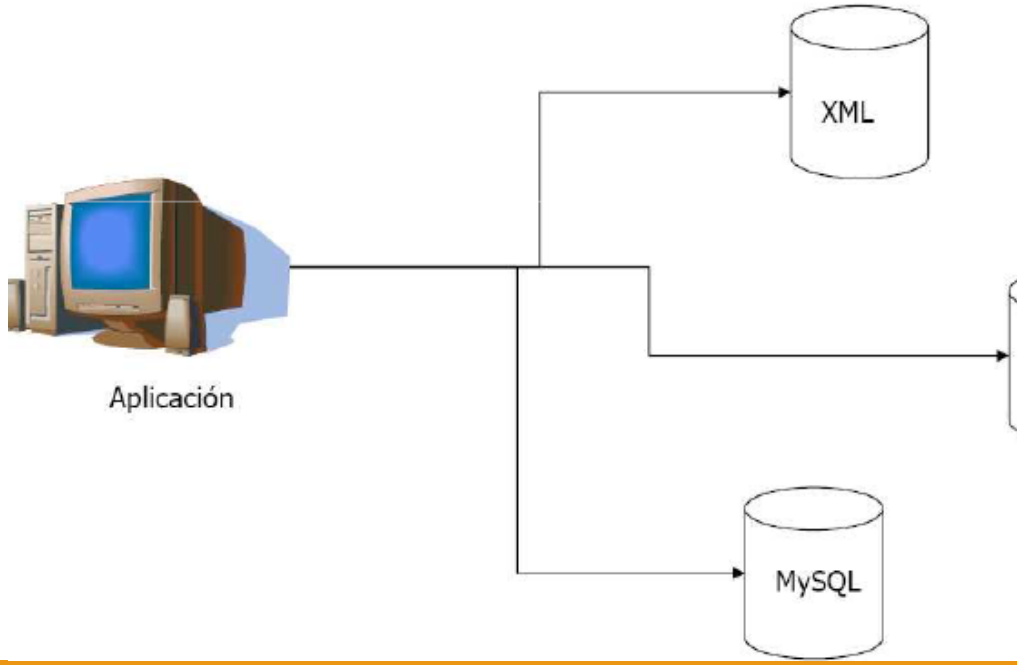


Arquitectura en Capas - Patrón DAO

- Problema:
  - Se necesita independencia del sistema de persistencia
    - BD Relacional, BD orientada a objetos, ficheros, XML, BD XML, serialización …
  - Se debe acceder a varios sistemas

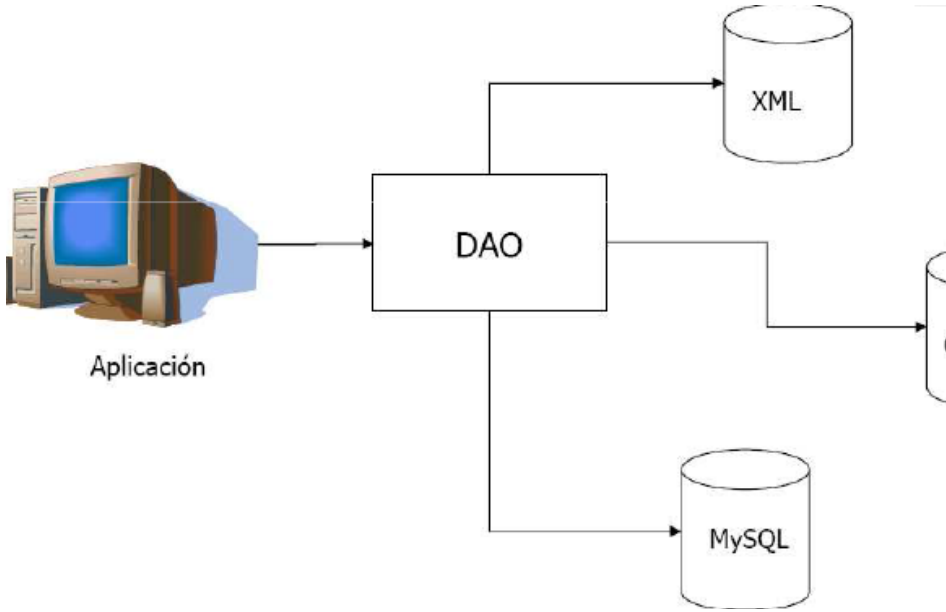
desde la misma aplicación

  - Las APIS son diferentes



Arquitectura en Capas - Patrón DAO

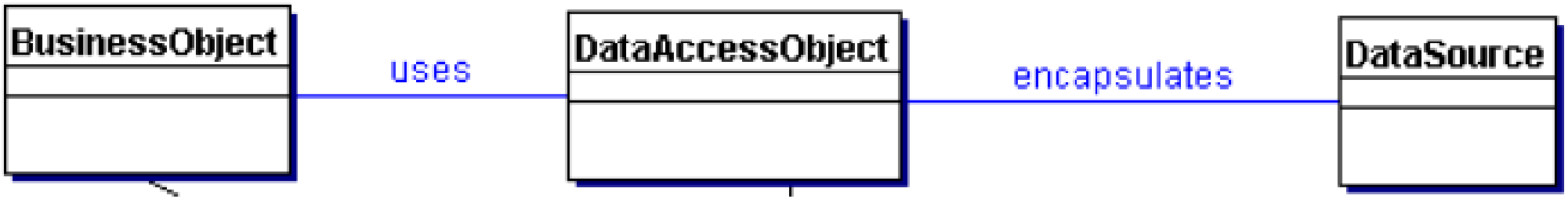
- Solución:
  - DAO: Data Access Object proporciona una interfaz única de acceso a los datos independientemente de cómo estén almacenados
  - Proporciona independencia a la lógica del negocio del acceso a datos
  - Habitualmente un objeto DAO por cada entidad del modelo



ver paquete libreria del tema 3

Patrón DAO - Implementaciones

- Objetos de acceso a datos
  - Clases DAO con JDBC y SQL
- Frameworks de persistencia mapeo O/R (objeto/relación)
  - Hibernate, Spring, JPA (Java Persistence API)



Actividad

- Para practicar
  - Ejercicios3.pdf

Referencias

1. Tutoriales oficiales de Oracle, http://docs.oracle.com/javase/tutorial/jdbc/overview/index.html
2. API de Java, http://docs.oracle.com/javase/8/docs/api/